QueryBloom: Automated Text-SPARQL Example Synthesis via Subgraph Sampling

Essam Wisam and Chengkai Li
The University of Texas at Arlington
Arlington, TX, USA
{abdelghanye, cli}@uta.edu

Abstract

Text-SPARQL example sets are indispensable for training, tuning, and evaluating Knowledge Graph Question Answering (KGQA) systems, yet they remain scarce for various knowledge graphs due to the extent of manual effort required. We present *QueryBloom*, an automated, KG-agnostic methodology that synthesizes diverse, comprehensive and high-quality Text-SPARQL examples requiring only a SPARQL endpoint for the target knowledge graph. The approach systematically samples structurally diverse multi-hop subgraphs from the knowledge graph, before employing an LLM-driven multi-agent framework to synthesize grounded text–SPARQL pairs and progressively refine them for complexity, correctness, and fluency. Evaluation against the human-authored DBpedia QALD-9 benchmark demonstrates that QueryBloom matches or exceeds established benchmarks in query structural complexity, semantic linguistic variety, and schema coverage.

1 Introduction

RDF knowledge graphs (KGs) have become a standard framework for structuring open-domain information, as exemplified by knowledge bases such as DBpedia and Wikidata [3, 23]. Simultaneously, their application has expanded to capture complex, domain-specific knowledge in areas ranging from soil carbon and climate modeling to medicine and sociology [14]. SPARQL has become the standard query language for RDF knowledge graphs, with hundreds of queryable endpoints emerging since its inception [2]. Owing to the technical barrier of composing SPARQL queries, Knowledge Graph Question Answering (KGQA) systems have become essential for democratizing access to knowledge graphs, enabling researchers, domain-experts, and ordinary users to seamlessly reason about such knowledge by mapping natural language questions to SPARQL queries that can be executed against the graph to provide accurate answers.

discussing KGQA in the context of Ai.

the users are not just researchers. it could be domain experts, non experts, and agentic systems too.

Text-SPARQL example sets play a central role for KGQA systems. They serve as in-context demonstrations that steer LLM-based query generation [26, 28, 6, 9], as training corpora for fine-tuning translation models [15, 27], and as standardized benchmarks for measuring system correctness [8, 21, 22]. Beyond training and evaluation, these example sets are often used as executable documentation, helping human practitioners better navigate unfamiliar knowledge graph structures and query idioms [20, 25].

More systematic exploration of the distinction between 3 natural language query formats: keywords, questions, and commands.

Constructing high-quality Text-SPARQL example sets for a given knowledge graph is a costly and time-consuming process because it requires three distinct areas of expertise: (i) fluency in the specific target domain, (ii) proficiency in formal SPARQL query construction, and (iii) a deep familiarity with the knowledge graph's ontology, including its semantics, structure, and vocabulary. Existing curated Text-SPARQL benchmarks concentrate on a handful of prominent open-domain knowledge graphs, principally DBpedia, Wikidata, and Freebase, and rely on manual question authoring, template-based construction, and contributions from the open-source community [21, 22, 19, 8, 26, 12]. Domain-specific knowledge graphs, which encode specialized knowledge in fields such as biomedicine [5], geoscience [16], and climate models [1], lack equivalent community infrastructure and consequently have few or no Text-SPARQL resources [14]. This scarcity has confined the majority of KGQA research [12, 28, 15, 10, 4, 7, 18, 26] to the open-domain settings where Text-SPARQL example sets already exist, leaving domain-specific settings largely unaddressed, and depriving domain researchers of the automated QA interfaces necessary to seamlessly reason over specialized knowledge embedded in domain-specific knowledge graphs.

Make it clear that QALD and so on are such existing benchmarks on open-domain KGs.

Unclear why we discuss each benchmark's creation

To address this gap, there is a need for an automated KG-agnostic methodology that can generate

reliable, diverse and comprehensive Text-SPARQL pairs for arbitrary knowledge graphs, which to the best of our knowledge no prior work has yet addressed. To bridge this gap, we present QueryBloom, an automated, KG-agnostic methodology that can synthesize high-quality Text-SPARQL example sets from any RDFS-based knowledge graph given only SPARQL endpoint access. The approach is motivated by the two following observations: (i), it is possible to discover arbitrary subgraph patterns for any given RDFS-based knowledge graph using KG-agnostic RDFS-based exploration queries (Subgraph Discoverability) and (ii), every SPARQL query implicitly specifies a subgraph pattern over the knowledge graph (SPARQL-Subgraph Duality). QueryBloom inverts the duality by using an LLM agentic pipeline to generate Text-SPARQL queries grounded on subgraph instances and patterns automatically discovered from the target knowledge graph.

Concretely, the QueryBloom pipeline proceeds in four stages: (1) extracting one-hop subgraph type patterns encoding which properties connect which types of entities; (2) growing a set of structurally diverse multi-hop subgraphs by iteratively extending seed patterns through adjacency-guided attachment; (3) enriching each subgraph with concrete entity instances and attribute values; and (4) employing an agentic pipeline that filters non-meaningful subgraphs, jointly generates natural language questions and SPARQL queries, augments the pairs to introduce meaningful complexity, iteratively verifies each pair against the endpoint, and finally refines phrasing for the generated questions.

In summary, our key contributions are as follows:

- QueryBloom, an LLM-driven methodology that synthesizes diverse, accurate and, comprehensive Text-SPARQL examples for a target knowledge graph given only its SPARQL endpoint. It tackles the example scarcity challenge for domain-specific knowledge graphs and facilitates the development of their KGQA systems.

- A comprehensive evaluation framework demonstrating that QueryBloom-generated datasets match or exceed the diversity of established human-authored benchmarks across query structural complexity, semantic linguistic variety, and holistic schema coverage.

- Three novel Text-to-SPARQL datasets across diverse domain scopes: open-domain (DBPEDIA), broad-domain (DBLP), and domain-specific (SOCKG).

2 Related Work

Text-SPARQL Datasets. Several curated benchmarks have driven progress in KGQA. The QALD challenge series [21, 22] provides hand-crafted questions over DBpedia and Wikidata, while LC-QuAD [19] and LC-QuAD 2.0 [8] use template-based generation with human paraphrasing to scale dataset size. WikiSP [26] sources questions from search logs to better reflect real user information needs. Spinach [12] curates questions from Wikidata's "Request a Query" forum, capturing authentic complexity beyond synthetic benchmarks. All of these resources target open-domain knowledge graphs such as Wikidata, DBpedia, and Freebase; domain-specific graphs, which lack equivalent breadth or community efforts, have no comparable Text-SPARQL coverage.

KGQA via NL-SPARQL Translation. KGQA has seen significant advances with the emergence of Large Language Models (LLMs), which have achieved state-of-the-art performance compared to prior deep learning and classical approaches [6, 15]. LLM-based approaches include chain-of-thought prompting for SPARQL generation [28], fine-tuning on text-SPARQL pairs [15, 27], and in-context learning with retrieved examples [9]. Agentic approaches such as GRASP [24] and Spinach [12] employ LLMs with tool-use for interactive graph exploration. All of these approache require or at least benefit from Text-SPARQL examples. QueryBloom is thus, complementary to these systems, as it can automatically provide the example sets they require for arbitrary RDFS-based knowledge graphs.

Synthetic Data Generation. Template-based approaches such as LC-QuAD [19] generate questions from fixed SPARQL templates, but require manual template design per knowledge graph. Labate et al. [11] recently proposed using LLMs to replace crowdsourced paraphrasing of LC-QuAD's template-generated questions, showing that synthetic rephrasings can match human quality for fine-tuning; however, their method still depends on LC-QuAD's existing template infrastructure and is limited to Wikidata. Deviating from template-based generation, KGQAGen [29], employs LLMs

Need to articulate the motivations behind the methodology design before you detail the methods.

Explain the evaluation, summarize the results, and other aspects before the "summary".

can have more bullet points.

Are there related work in terms of methodology?

It seems obvious that they cannot be transferred.

you mentioned these are not fully accurate.

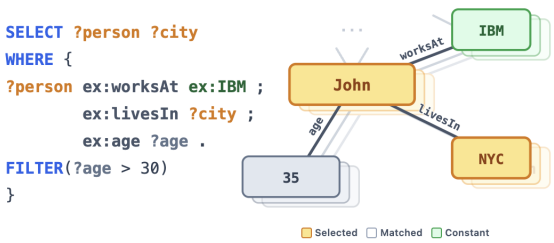


Figure 1: Correspondence between a SPARQL query and its matched knowledge graph subgraph.

to generate at most one Wikidata-specific Text-SPARQL pair for each seed entity drawn from Wikipedia vital articles with the aim of improving factual quality issues observed in earlier benchmarks. While similar to QueryBloom in generating the Text-SPARQL pairs grounded on knowledge embedded in the graph, their method is designed specifically for Wikidata and uses Wikidata-specific APIs. Moreover, it generates questions strictly bounded by the number of seed entities given. In contrast, QueryBloom is designed to be applicable on any RDFS-based knowledge graph given its SPARQL endpoint and comprehensively generates Text-SPARQL pairs covering the schema underlying the knowledge graph while optionally requiring a single seed for the whole knowledge graph. The two methods could be viewed as complementary because Wikidata does not follow the RDFS standard required by QueryBloom and offers various resources, including seed entities and APIs, that make KGQAGen possible. In contrast, QueryBloom is designed to operate on any RDFS-based knowledge graph and does not assume access to such specific resources.

3 Methodology

The central observation underlying QueryBloom is that every SPARQL query implicitly specifies a subgraph pattern over the knowledge graph (Figure 1). The triple patterns in a `WHERE` clause define nodes and labeled edges that must be matched in the graph, while `SELECT` projects specific variables, and `FILTER` imposes value constraints. Each returned result row corresponds to a distinct embedding of this pattern in the graph data: a concrete assignment of entities to the query's variables that satisfies all structural and value constraints. The same argument applies to other SPARQL query forms (`ASK`, `CONSTRUCT`), which rely on the same `WHERE` clause pattern matching mechanism.

QueryBloom inverts this correspondence: it first

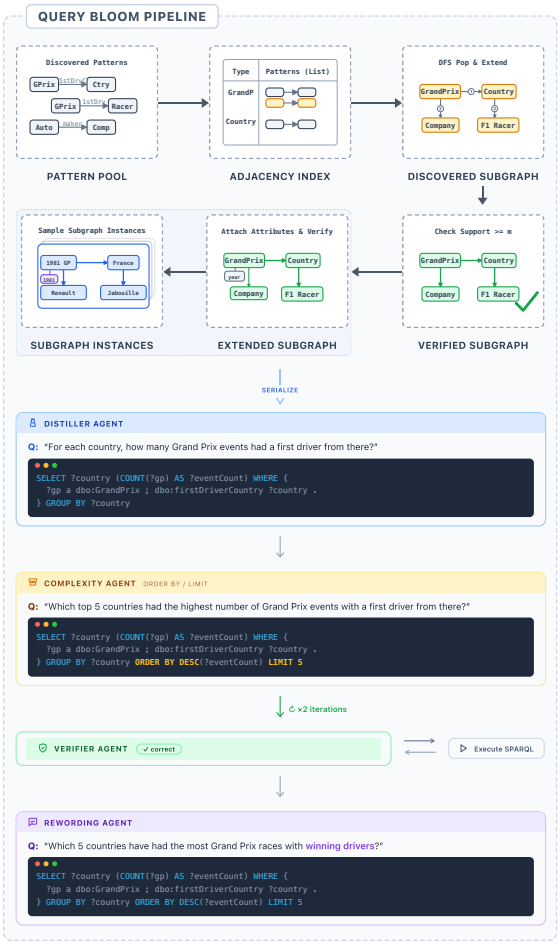


Figure 2: Overview of the QueryBloom methodology.

discovers structurally diverse subgraph patterns from the knowledge graph and then jointly synthesizes SPARQL queries and natural language questions grounded in those patterns (Figure 2).

The pipeline proceeds through the multiple stages described below, which trace how a single subgraph pattern is discovered, grounded, and distilled into a small set of text-SPARQL pairs. The framework relies on the minimal assumption that the target knowledge graph follows W3C RDFS standards for defining classes, which is a common practice in almost all RDF knowledge graphs.

3.1 Pattern Discovery and Adjacency Indexing

The pipeline begins by extracting *frequent type patterns* from a property scope chosen for the target knowledge graph, adapting the ABSTAT algorithm [17] for efficient pattern extraction over RDF graphs. A type pattern is a triple (*SubjectType*, *Predicate*, *ObjectType*) encoding that a given property connects instances of one type to instances of another (e.g., `Person –worksAt –`

`Organization`); patterns are discovered by sampling a large set of triples for each property and resolving the `rdf:type` of their subject and object nodes.

QueryBloom supports two preprocessing modes that differ in how the property scope is determined. *Whole-graph mode*, suited to smaller knowledge graphs, enumerates all properties via a SPARQL query, removes common metadata predicates such as labels and identifiers, and extracts patterns from the resulting set. *Online-graph mode*, suited to large graphs, begins from a single user-specified seed property, extracts its type patterns, and then iteratively selects properties connected to already-discovered types, adding their patterns until a configurable property budget is reached.

In both modes, the collected patterns are filtered by a minimum support threshold σ; the resulting set of patterns form the structural building blocks for subgraph construction. An adjacency index is then built over this pool, mapping each type to every pattern in which it participates and enabling constant-time lookup of candidate extensions during subgraph discovery.

3.2 Subgraph Discovery

A multi-hop subgraph pattern is grown from a single frequent pattern, which serves as a two-type seed. The seed is iteratively extended by attaching adjacent patterns from the adjacency index that share at least one type with the current subgraph. Each extension adds a new edge and may introduce new types, continuing until a subgraph with k distinct types is formed. At each extension, the subgraph's support is verified via a SPARQL `COUNT` query and the extension is discarded if it causes the subgraph's support to fall below the threshold σ. It follows that the subgraph will contain at least $k-1$ properties and may be cyclic or acyclic.

Once a k-type subgraph is formed, it is enriched in two further steps. First, data attributes are sampled from the discovered attribute patterns and attached to the subgraph's types and verified. Second, m concrete entity instances per type that jointly satisfy the subgraph's type and property constraints are fetched from the endpoint, along with their attribute values. The subgraph comprises the abstract subgraph pattern and grounding instances with real entity URIs, labels, and attribute values to inform realistic question generation.

3.3 LLM-Driven Text-SPARQL Distillation

Each verified subgraph is processed by a four-stage multi-agent system whose collective purpose is to distill the subgraph into a small set of high-quality, execution-verified text-SPARQL pairs. The four agents — a *generator*, a *complexity introducer*, a *verifier*, and a *reword auditor* — operate in sequence, each receiving the subgraph as a structured YAML representation. All agents use structured output with enforced JSON schemas to guarantee perfectly parseable responses [13].

Generation. The generator agent first assesses whether the subgraph pattern admits meaningful questions about the knowledge graph, exiting early if not. If meaningful, it generates a set of text-SPARQL pairs grounded in the subgraph's classes, entities, relationships, and attributes (see Appendix A). Each generated SPARQL query is immediately executed against the endpoint, and its results are recorded alongside the pair.

Complexity Introduction. To ensure that generated question–query pairs match the complexity of real-world examples, a *complexity introducer* agent rewrites each successfully executed pair to meaningfully incorporate additional non-trivial constructs drawn from a predefined set: value filtering, optional matching, set union, sorting and limiting, existential checks, nested subqueries, and recursive relationships. Both the SPARQL query and its corresponding natural language question are rewritten consistently to reflect the introduced construct. Where no meaningful augmentation is possible, the agent skips the pair and retains the original. Every augmented query is re-executed against the endpoint to update the execution results.

Verification. Although grounding generation in a concrete subgraph pattern is in principle sufficient to produce accurate text-SPARQL pairs (Section 3), an LLM is still prone to hallucination or innovation while forming the text-SPARQL. An iterative verifier agent therefore inspects every pair. Given the subgraph pattern and each pair's question, query, and execution results, the verifier judges each pair as *correct* or *needs-fix*. For pairs it deems in need of fixing, it additionally produces corrected question and query texts; the corrected queries are re-executed and the updated pairs are fed back into the next verification round. The loop runs for up to a configurable number of iterations or until all pairs are marked correct.

Question Rewording. As a final step, a reword agent audits the natural language question of each correct pair for KG-specific terminology leakage — such as exposed URI fragments, ontology class names, or other non-natural phrasings — and rewrites only the question text of flagged pairs, leaving the SPARQL query unchanged.

All four LLM calls operate with high reasoning effort to maximise correctness, and only pairs that pass verification are retained in the final dataset.

3.4 Design Principles

The pipeline is designed to produce a text-SPARQL example set that is *comprehensive*, *diverse*, and of *high quality*.

Comprehensiveness. The overall algorithm orchestrates the stages described above by seeding at least one subgraph from every frequent type pattern in the constructed pool. Thus, coverage is defined with respect to the chosen preprocessing mode: whole-graph mode covers all frequent patterns found among the graph's selected non-metadata properties, whereas online graph mode covers all frequent patterns found for the seed property and any connected properties added within the property budget. As opposed to graph mining, we deliberately adopt a very low support threshold because even rare subgraph patterns can give rise to meaningful questions.

Diversity. Several mechanisms ensure diversity in the generated examples: (i) unlike a random walk that follows a linear path, the subgraph is extended by selecting a new pattern from *any type* contained in the current subgraph, producing diverse structural compositions; (ii) the target number of types k is sampled from a range distribution to produce subgraphs of varying sizes; (iii) for each subgraph, we randomly sample v attributes per class to ensure diverse coverage of class attributes; (iv) the distiller agent is prompted to produce variations in question and query style, covering different query patterns (e.g., `SELECT`, aggregation, `ASK`, `FILTER`, `OPTIONAL`, `GROUP BY`).

Quality. Quality is safeguarded at multiple stages. Subgraphs that are trivial or concern only graph metadata are flagged as non-meaningful during generation and excluded. The generate-execute-verify loop ensures that every retained pair produces results expected by the distiller agent. The distiller agent's core objective is to elicit questions from the subgraph that a domain expert would be expected to ask.

4 Results

4.1 Dataset Quality and Diversity

To assess the quality and structural diversity of datasets generated by QueryBloom, we compare a synthesized dataset against DBpedia QALD-9, a standard human-authored benchmark. We evaluate diversity across three dimensions: query feature complexity, semantic linguistic variety, and holistic schema coverage.

Query Structural Complexity. The distribution of SPARQL 1.1 operators within both datasets shows that QueryBloom demonstrates significantly higher structural complexity in several key dimensions compared to the QALD-9 baseline. Notably, QueryBloom queries incorporate `FILTER` constraints at roughly four times the rate of the baseline. Furthermore, they heavily utilize `OPTIONAL` blocks and property paths, which are entirely absent from the QALD-9 test set. Both datasets maintain a comparable usage of `UNION` operators. This confirms that QueryBloom maintains a higher level of structural complexity than the QALD-9 baseline which more closely reflects the complexity of real-world SPARQL queries.

Semantic Linguistic Diversity. To evaluate the natural language variety of the generated questions, we compute dense semantic embeddings for each question text using SBERT (BGE-large) and project these embeddings into a 2D space via t-SNE. We compute Average Pairwise Distance (APD) for global diversity and Mean Nearest-Neighbour Distance (MNND) for local density. QueryBloom achieves a higher APD and MNND, which indicates that QueryBloom successfully covers a wider semantic space of linguistic intents and vocabulary than the fixed human-authored QALD-9 test set.

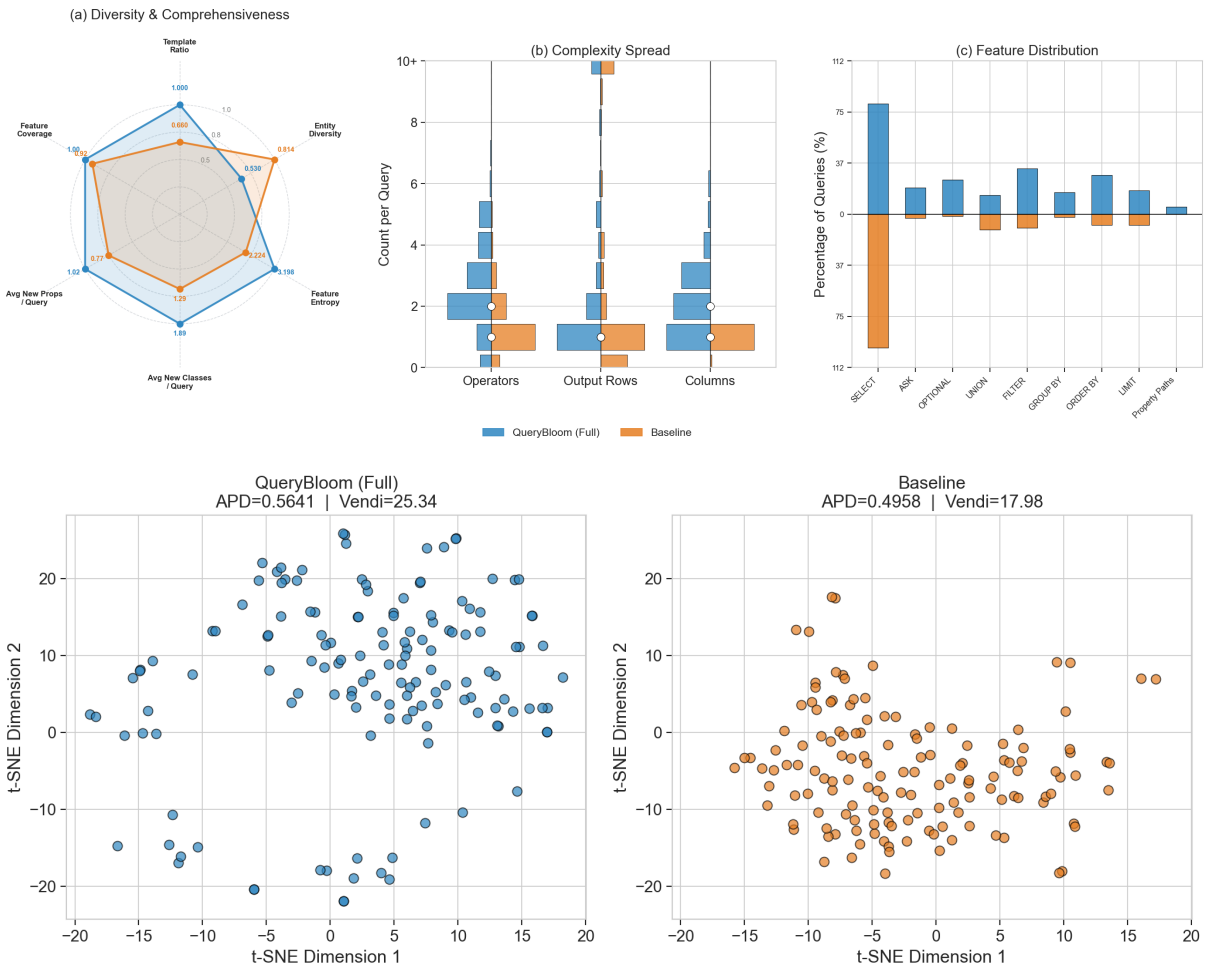
Holistic Schema Coverage. We present a multi-dimensional summary of dataset diversity across five key metrics:

- **Unique Entities & Relations:** The average number of distinct entities and properties referenced per query, reflecting how broadly the dataset explores the underlying knowledge graph schema.
- **Variables & Triples:** The median number of variables and triple patterns per query, mea-

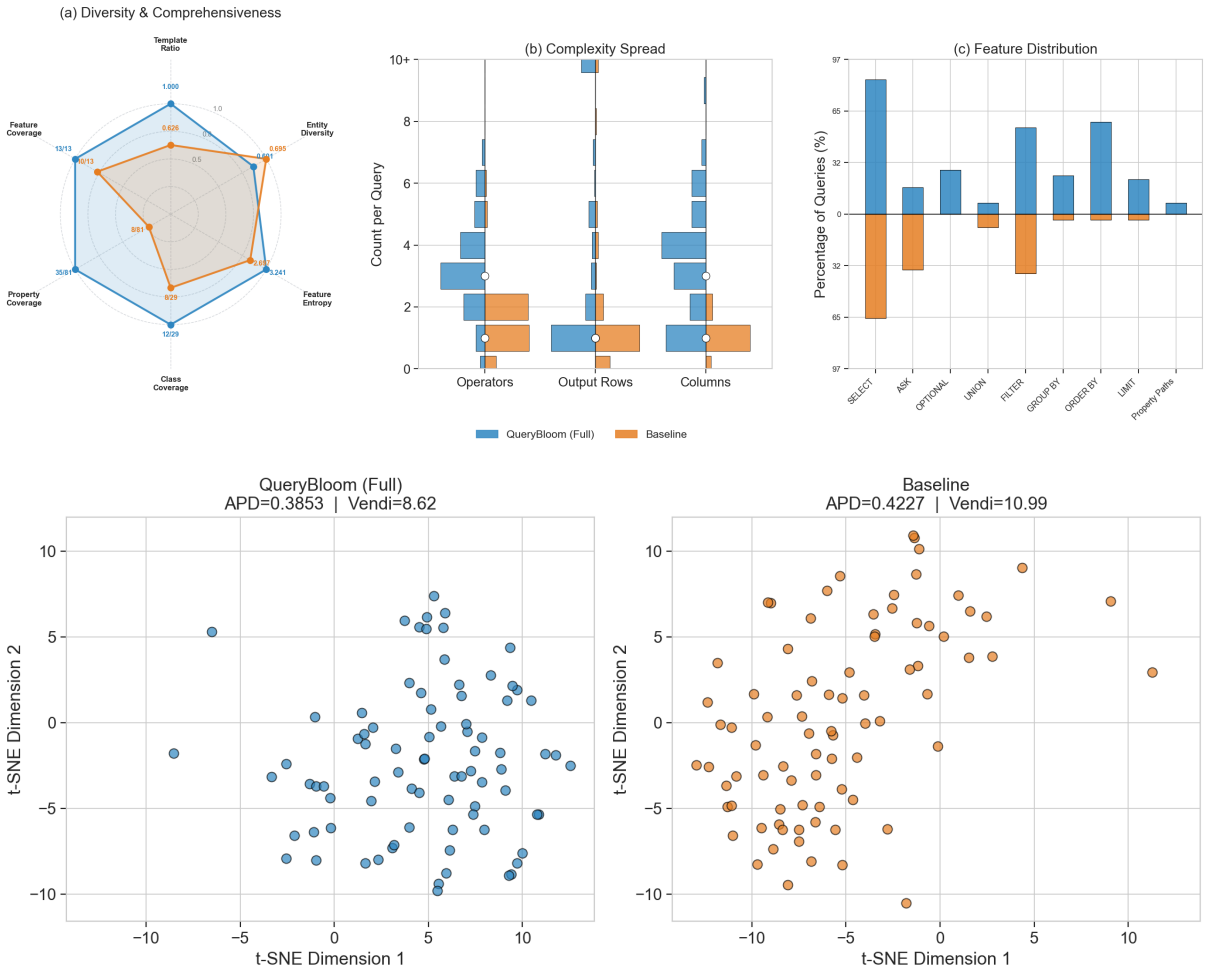
suring the structural size and depth of the generated SPARQL.

- **Unique Templates:** The ratio of distinct SPARQL skeletons (where specific entities, variables, and literals are anonymized) to the total number of queries, assessing syntactic novelty.

QueryBloom outperforms QALD-9 across almost all these dimensions. Most strikingly, it achieves a substantially higher unique template ratio, indicating that its SPARQL skeletons are even more diverse than those found in the human-authored benchmark. The next clearest difference is structural depth, with QueryBloom producing a median of roughly three times as many triple patterns per query as the baseline. Furthermore, it consistently incorporates a higher rate of unique entities per query, ensuring broader and more diverse coverage of the graph's instance data.



DBpedia knowledge graph



DBLP knowledge graph

5 Conclusion

We presented QueryBloom, a KG-agnostic methodology for automatically synthesizing diverse, high-quality Text-SPARQL example sets from arbitrary RDF knowledge graphs, requiring only SPARQL endpoint access. Through multi-hop subgraph sampling and LLM-driven verbalization with execution-based verification, QueryBloom produces example sets that match or exceed the diversity of established human-authored benchmarks across query structural complexity, semantic linguistic variety, and schema coverage. By addressing the example scarcity challenge for domain-specific knowledge graphs, QueryBloom can support the development and evaluation of KGQA systems in specialized domains where curated resources are unavailable, while also serving as executable documentation that helps practitioners understand unfamiliar graph structures.

References

[1] Mustapha Adamu, Qi Zhang, Huitong Pan, Longin Jan Latecki, and Eduard C. Dragut. 2025. Querying climate knowledge: Semantic retrieval for scientific discovery.

[2] Waqas Ali, Muhammad Saleem, Bin Yao, Aidan Hogan, and Axel-Cyrille Ngonga Ngomo. 2022. A survey of RDF stores & SPARQL engines for querying knowledge graphs. *The VLDB Journal*, 31(3):1–26.

[3] Sören Auer, Christian Bizer, Georgi Kobilarov, Jens Lehmann, Richard Cyganiak, and Zachary Ives. 2007. DBpedia: A nucleus for a web of open data. In *The Semantic Web*, pages 722–735. Springer.

[4] Debayan Banerjee, Pranav Ajit Nair, Jivat Neet Kaur, Ricardo Usbeck, and Chris Biemann. 2022. Modern baselines for SPARQL semantic parsing. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2260–2265.

[5] Knowledge Graphs Consortium. 2025. An update on knowledge graphs and their applications in computational drug discovery. *Expert Opinion on Drug Discovery*.

[6] Jacopo D'Abramo, Andrea Zugarini, and Paolo Torroni. 2025. Investigating large language models for text-to-SPARQL generation. In *Proceedings of the 4th International Workshop on Knowledge-Augmented Methods for Natural Language Processing*, pages 66–80.

[7] Papa Abdou Karim Karou Diallo, Samuel Reyd, and Amal Zouaq. 2025. FRASE: Structured representations for generalizable SPARQL query generation. *Preprint*, arXiv:2503.22144.

[8] Mohnish Dubey, Debayan Banerjee, Abdelrahman Abdelkawi, and Jens Lehmann. 2019. LC-QuAD 2.0: A large dataset for complex question answering over Wikidata and DBpedia. In *Proceedings of the 18th International Semantic Web Conference*, pages 69–78.

[9] Vincent Emonet, Jerven Bolleman, Severine Duvaud, Tarcisio Mendes de Farias, and Ana Claudia Sima. 2024. LLM-based SPARQL query generation from natural language over federated knowledge graphs. In *Proceedings of the Special Session on Harmonising Generative AI and Semantic Web Technologies (HGAIS 2024) co-located with ISWC 2024*, volume 3953 of *CEUR Workshop Proceedings*, pages 1–10.

[10] Endri Kacupaj, Joan Plepi, Kuldeep Singh, Harsh Thakkar, and Jens Lehmann. 2023. SPARQLGEN: Generating SPARQL queries from natural language questions. In *Proceedings of the ACM Web Conference 2023*, pages 1131–1141.

[11] Anton Bulle Labate, Breno William Santos Rezende de Carvalho, Sandro Rama Fiorini, and Viviane Torres da Silva. 2025. Synthetic dataset generation and validation with LLMs for text2sparql in Wikidata graphs. In *Proceedings of the 5th Wikidata Workshop (Wikidata 2025)*, volume 4108 of *CEUR Workshop Proceedings*.

[12] Shicheng Liu, Sina Semnani, Harold Triedman, Jialiang Xu, Isaac Dan Zhao, and Monica Lam. 2024. SPINACH: SPARQL-based information navigation for challenging real-world questions. In *Findings of the Association for Computational Linguistics: EMNLP 2024*, pages 15977–16001.

[13] OpenAI. 2024. Introducing structured outputs in the api.

[14] RENCI. 2024. FRINK: The fabric integrating networked knowledge (proto-OKN theme 1). Website: `https://frink.renci.org/registry/`.

[15] Md Rashad Al Hasan Rony, Uttam Kumar, Roman Teucher, Liubov Kovriguina, and Jens Lehmann. 2022. SGPT: A generative approach for SPARQL query generation from natural language questions. *IEEE Access*, 10:70712–70723.

[16] Nasim Shirvani-Mahdavi, Devin Wingfield, Juan Guajardo Gutierrez, Mai Tran, Zhengyuan Zhu, and Zeyu Zhang. 2025. A knowledge graph informing soil carbon modeling. In *Proceedings of the 25th International Conference on Web Engineering*, pages 226–241.

[17] Blerina Spahiu, Riccardo Porrini, Matteo Palmonari, Anisa Rula, and Andrea Maurino. 2016. Abstat: Ontology-driven linked data summaries with pattern minimalization. In *SumPre@ESWC*.

[18] Jiashuo Sun, Chengjin Xu, Lumingyuan Tang, Saizhuo Wang, Chen Lin, Yeyun Gong, Lionel M. Ni, Heung-Yeung Shum, and Jian Guo. 2024. Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. In *Proceedings of the Twelfth International Conference on Learning Representations*.

[19] Priyansh Trivedi, Gaurav Maheshwari, Mohnish Dubey, and Jens Lehmann. 2017. LC-QuAD: A corpus for complex question answering over knowledge graphs. In *Proceedings of the 16th International Semantic Web Conference*, pages 210–218.

[20] UniProt Consortium. 2025. UniProt SPARQL query examples. Accessed: 2025-08-27.

[21] Ricardo Usbeck, Ria Hari Gusmita, Axel-Cyrille Ngonga Ngomo, and Muhammad Saleem. 2018. 9th challenge on question answering over linked data (QALD-9) (invited paper). In *Joint Proceedings of the 4th Workshop on Semantic Deep Learning (SemDeep-4) and NLIWoD4: Natural Language Interfaces for the Web of Data (NLIWOD-4) and 9th Question Answering over Linked Data*

Challenge (QALD-9) co-located with 17th International Semantic Web Conference (ISWC 2018), volume 2241 of *CEUR Workshop Proceedings*, pages 58–64. CEUR-WS.org.

[22] Ricardo Usbeck, Xi Yan, Aleksandr Perevalov, Longquan Jiang, Julius Schulz, Angelie Kraft, Cedric Möller, Junbo Huang, Jan Reineke, Axel-Cyrille Ngonga Ngomo, Muhammad Saleem, and Andreas Both. 2024. QALD-10 - the 10th challenge on question answering over linked data: Shifting from DBpedia to Wikidata as a KG for KGQA. *Semantic Web*, 15(6):2193–2207.

[23] Denny Vrandečić and Markus Krötzsch. 2014. Wikidata: A free collaborative knowledgebase. *Communications of the ACM*, 57(10):78–85.

[24] Sebastian Walter and Hannah Bast. 2025. GRASP: Generic reasoning and SPARQL generation across knowledge graphs. In *Proceedings of the 24th International Semantic Web Conference*, pages 271–289.

[25] Wikidata Contributors. 2025. Wikidata SPARQL query service: Examples. Accessed: 2025-08-27.

[26] Silei Xu, Shicheng Liu, Theo Culhane, Elizaveta Pertseva, Meng-Hsi Wu, Sina Semnani, and Monica Lam. 2023. Fine-tuned LLMs know more, hallucinate less with few-shot sequence-to-sequence semantic parsing over Wikidata. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5778–5791.

[27] Xiaoyu Yin, Dagmar Gromann, and Sebastian Rudolph. 2021. Neural machine translating from natural language to SPARQL. *Future Generation Computer Systems*, 117:510–519.

[28] Hamada Mohamed Abdelsamee Zahera, Ali Manzoor, Mohamed Sherif, Diego Moussallem, and Axel-Cyrille Ngonga Ngomo. 2024. Generating SPARQL from natural language using chain-of-thoughts prompting. In *Knowledge Graphs in the Age of Language Models and Neuro-Symbolic AI - Proceedings of the 20th International Conference on Semantic Systems, 17-19 September 2024, Amsterdam, The Netherlands*, Studies on the Semantic Web, pages 353–368. IOS Press.

[29] Liangliang Zhang, Zhuorui Jiang, Hongliang Chi, Haoyang Chen, Mohammed Elkoumy, Fali Wang, Qiong Wu, Zhengyi Zhou, Shirui Pan, Suhang Wang, and Yao Ma. 2025. Diagnosing and addressing pitfalls in kg-rag datasets: Toward more reliable benchmarking. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*.

A Prompts

A.1 QueryBloom Generation Prompt

Structure of the Generation Prompt

```
You are an expert in analyzing a subgraph sampled from a knowledge graph and generating natural language questions with
      corresponding SPARQL
queries.


{% if kg_title %}
**Knowledge Graph Title:** {{ kg_title }}
{% endif %}
{% if kg_description %}
**Knowledge Graph Description:** {{ kg_description }}
{% endif %}

Given the following subgraph description loaded from a YAML file, generate
natural language questions that could be answered by querying the full knowledge graph, along with their corresponding
SPARQL queries.

### YAML Subgraph Description:
<Subgraph>
   {{ subgraph_pattern }}
</Subgraph>

Notes on the YAML structure (generic, do not assume any specific KG):
- Each subgraph includes: subgraph_id, num_types, min_support, types, edges, predicate_uris.
- types is a mapping from type label to:
- uri (class URI)
- sample_entities (a few example entity URIs, sometimes with attributes shown as a compact string like attr:type=value)
- edges is a list of triples in a readable form: "TypeA --predicateLabel --> TypeB".
- if an edge ends with "\*" after the arrow, it denotes an optional property (after adding the edge, the graph doesn't
meet the support property).
- predicate_uris lists predicate URIs aligned with the edges.
- Attributes marked with "\*" are below the subgraph's support threshold (optional attributes).

### Task:
First, assess whether this subgraph allows for meaningful questions. If the subgraph is too trivial, lacks interesting
relationships (eg, graph metadata as opposed to actual knowledge), or doesn't support generating useful and meaningful
      questions, set is_meaningful to false and return an empty questions
list.

If is_meaningful is true, generate 2-4 question-query pairs as a JSON array where each item has:
- text_query: the natural language question
- sparql_query: the corresponding SPARQL query



### Requirements:
1. Generate 2-4 question-query pairs depending on your assessment of the subgraph's richness. Try to attain high coverage
       of all the meaningful aspects of the subgraph in the question-query pairs.
2. Questions should be grounded in the classes, entities, relationships and attributes in this YAML subgraph. Using
      classes, entities, relationships and attributes not in the YAML subgraph is strictly prohibited!
3. Do not reference the YAML or the subgraph itself in the natural language question (each of text and query should be
      self-contained) and never mention a URI/prefix in the text question (or use knowledge graph terminology like
      properties, types, etc.)
4. Questions should be certaintly answerable and avoid ontology/schema/documentation questions
5. Focus on questions that are interesting and meaningful (avoid meaningless questions like "what/how many entities are
      linked to.../useless patterns...")
{% if focus_entity_qa %}
6. Great questions are often centered around interesting entities in the graph. All of the questions you produce must be
      entity centric.
Meanwhile, set is_meaningful to False if no interesting entities are available.
7. Make questions diverse in style (what, who, whom, whose, when, which, yes/no questions, how compounds, statistics, etc
      .) and complexity but while maintaining the quality of the questions and meaningfulness
{% else %}
6. Whenever there is an interesting entity is in the graph, it's good to have a question or two specifically about it
7. Make questions diverse in style (SELECT, SELECT + AGGREGATION, ASK, as appropriate) and complexity (MIN/MAX/AVG/COUNT/
      SUM, GROUP BY/FILTER/HAVING, EXISTS/OPTIONAL, nesting, etc.) but while maintaining the quality of the questions and
       meaningfulness
{% endif %}
8. Declare meaningful prefixes in the query based on the URIs in the yaml
9. Use proper SPARQL syntax to generate valid queries. Your sparql query must produce exactly the results expected by the
       text query.
10. If the question does not have one answer/result, you may or may not use LIMIT in SELECT queries (e.g., LIMIT 100) to
      avoid too many results. BUT MAKE SURE to REFLECT THIS in the text query!
11. SPARQL queries should be stand-alone executable against a SPARQL endpoint containing the full graph
12. If a term is awkward in the graph, and you are confident it can be rephrased naturally without being inaccurate, do
      so
13. The phrasing of the natural language question should be always natural, fluent and not awkward or stilted. Question
      should always end with a question mark.
```

```
### Final Note:
The ultimate goal is to produce high-quality, diverse, and very meaningful questions together with their corresponding
accurate SPARQL queries that can be executed against the full knowledge graph.

Another essential property is that an expert in the knowledge graph should be able to reproduce the same sparql query
      when given text question only.
- You will be penalized if the generated SPARQL query and the text question do not properly align (eg, the sparql query
      has constraints unrequested in the question... or misses requested ones)
- You will be also highly penalized if the generated SPARQL query incorrectly answer the text question or provides no
      answer when there is one.
- You will be penalized if your question styles are not diverse enough, if questions are of low quality or are phrased
      awkwardly.
- You will be rewarded when you generate questions an expert in the domain would ask where the sparql query being
      perfectly aligned to the question and accurately answering it.
- Flagging the subgraph as not meaningful when it's hard to generate high quality questions is better than generating low
       quality questions.
- Generating fewer high quality questions that a DOMAIN EXPERT may ask is MUCH better than generating more low quality
      questions. Quality over quantity.
```

A.2 QueryBloom Verification Prompt

Verification Prompt for Question–Query Pairs

```
You are an expert in SPARQL and knowledge graph querying. You are reviewing question-query pairs that were generated
from a subgraph pattern and then executed against a SPARQL endpoint.

Your task is to verify whether each question-query pair is correct based on the execution results.

### Original Subgraph Pattern:
<Subgraph>
    {{ subgraph_pattern }}
</Subgraph>

### Generated Question-Query Pairs with Execution Results:
{% for qa in qa_pairs %}
**Pair {{ loop.index }}:**
- **Question:** {{ qa.question }}
- **SPARQL Query:**
```sparql
{{ qa.sparql_query }}
```
- **Execution Success:** {{ qa.execution_results.success }}
- **Result Count:** {{ qa.execution_results.count }}
{% if qa.execution_results.error_message %}- **Error:** {{ qa.execution_results.error_message }}{% endif %}
{% if qa.execution_results.result_preview %}- **Result Preview (first 3 rows):** {{ qa.execution_results.result_preview
}}{% endif %}

{% endfor %}

Task:
For each question-query pair, assess whether it is **correct**, **needs_fix**, or **unfixable** based on:
1. Did the SPARQL query execute successfully?
2. Does the query correctly answer the natural language question? Are they consistent?
3. Are there any necessary fixes to the text or sparql queries.

Set `all_correct` to true only if ALL pairs are correct. If ANY pair needs fixing, set `all_correct` to false.

For pairs that need fixing:
- Provide a brief explanation in `reason`
- Provide the corrected question in `fixed_text_query`
- Provide the corrected SPARQL query in `fixed_sparql_query`

For correct pairs, leave `fixed_text_query` and `fixed_sparql_query` as null.

Important: a query that errors out, returns no results when it should, or doesn't match the question
semantics should be marked as `needs_fix`. Use `unfixable` only when no reliable correction exists.
```